Section 7: Pattern Selection Framework and Anti-Patterns

This section is part of the 5-hour LLD course by Mukul Raina

- YouTube Course: https://www.youtube.com/watch?v=VSABVEBDhf0
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 81: Section Intro: Pattern Selection Framework and Anti-Patterns

SLIDE 82: The Pattern Selection Process

SLIDE 82B: Step-by-Step: From Requirements to Patterns

SLIDE 83: Complete Signal-to-Pattern Mapping

SLIDE 84: Worked Example: E-Commerce Checkout System

SLIDE 84B: Worked Example: Chat Messaging System

SLIDE 85: Worked Example: File Storage and Processing Service

SLIDE 85B: Interview Practice: Rapid Pattern Recognition

SLIDE 86: Over-Engineering Anti-Pattern

SLIDE 86B: Minimum Viable Design Principle

SLIDE 87: Patterns Interviewers Test Frequently vs Rarely

SLIDE 87B: Pattern Testing Frequency by Company and Level

SLIDE 88: God Class Anti-Pattern

SLIDE 88B: God Class: Concrete Example and Refactoring

SLIDE 89: Feature Envy and Inappropriate Intimacy

SLIDE 90: Shotgun Surgery Anti-Pattern

SLIDE 91: Circular Dependencies

SLIDE 91B: Premature Abstraction

SLIDE 92: Interview Practice: Spotting Anti-Patterns in Code

SLIDE 93: How to Present Your Design in an Interview

SLIDE 93B: Structuring Your LLD Answer: Common Timing Mistakes

SLIDE 94: Note It Down: Complete Pattern Selection Reference

SLIDE 95: Section 7 Interview Practice and Common Mistakes

SLIDE 96: Section Summary and Key Takeaways

SLIDE 81: Section Intro: Pattern Selection Framework and Anti-Patterns

Section 7: Pattern Selection Framework and Anti-Patterns

Knowing individual design patterns is necessary but not sufficient for LLD interviews. The real skill is recognizing which pattern to apply when you hear a requirement, and equally important, recognizing when applying a pattern would make the design worse. This section builds a repeatable selection framework that maps requirement signals to patterns, walks through complete worked examples of multi-pattern selection, and covers the most common anti-patterns that interviewers test. Think of this section as the “pattern recognition” equivalent of the problem-pattern mapping you use in algorithm interviews.

What We Will Cover

1. The pattern selection process: a step-by-step approach
2. Complete signal-to-pattern mapping across all patterns from Sections 4-6
3. Three complete worked examples: from requirements to pattern selection
4. Over-engineering and minimum viable design
5. Pattern testing frequency by company and level
6. Six production anti-patterns: God Class, Feature Envy, Shotgun Surgery, Circular Dependencies, Premature Abstraction, Inappropriate Intimacy
7. How to structure and present your LLD answer in an interview

Why This Section Matters

Without This Section	With This Section
You know 14 patterns but freeze when given a new problem	You hear a requirement signal and instantly know which pattern applies
You apply patterns because you know them, not because they fit	You justify each pattern with a specific problem it solves
You over-engineer with 6 patterns when 2 would suffice	You apply the minimum set of patterns that solves the actual requirements
You cannot spot design flaws in your own or others' code	You recognize anti-patterns and name specific refactorings

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 82: The Pattern Selection Process

Pattern selection is not intuition. It is a repeatable process that you can apply systematically during an LLD interview. Here are the five steps.

Step 1: Identify the Objects and Their Responsibilities

Before reaching for any pattern, identify the core objects in the system and assign each a single responsibility. This is OOP fundamentals (Section 2) and SRP (Section 3). If your objects are wrong, no pattern will save the design.

Step 2: Identify the Dimensions of Variation

Ask: "What in this system changes or varies?" This is the most important question for pattern selection.

- If the algorithm varies: Strategy
- If the object's behavior varies by state: State
- If the reaction to an event varies: Observer
- If the object creation varies: Factory Method
- If the construction complexity varies: Builder

Step 3: Identify the Cross-Cutting Concerns

Ask: "What behaviors need to be added without modifying core logic?"

- Logging, caching, metrics, encryption: Decorator
- Access control, lazy loading: Proxy
- Interface translation for external systems: Adapter
- Multi-service coordination: Facade

Step 4: Check for Undo, Queuing, or Pipeline Needs

- Operations need to be undone or queued: Command
- Requests pass through sequential checks: Chain of Responsibility
- Algorithm structure is shared but steps differ: Template Method

Step 5: Validate Against the Requirements

For each pattern you selected, verify: "Does the requirement actually demand this pattern, or am I adding it because I know it?" If a simpler solution works (a plain interface, a simple method call, a constructor), prefer that. Patterns add indirection. Indirection is only justified when it solves a specific extensibility, decoupling, or maintainability problem.

Interview Relevance: Walking through these steps out loud during your interview shows the interviewer a structured thought process, not ad-hoc pattern guessing.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 82B: Step-by-Step: From Requirements to Patterns

Here is the five-step process applied to a concrete example.

Requirement: "Design a notification service that sends alerts through email, SMS, and push. Different notification types (marketing, transactional, alert) should be routed to different channels. When a notification is sent, log it for compliance and update analytics."

Step 1: Core Objects

`NotificationService` (orchestrator), `NotificationChannel` (delivery), `NotificationRouter` (routing logic), `ComplianceLogger` (logging), `AnalyticsTracker` (analytics).

Step 2: Dimensions of Variation

The delivery channel varies (email, SMS, push). This is a Strategy signal. The routing logic selects which channel to use. This is a Factory signal.

Step 3: Cross-Cutting Concerns

Compliance logging and analytics tracking are reactions to the "notification sent" event, not part of the core delivery logic. This is an Observer signal.

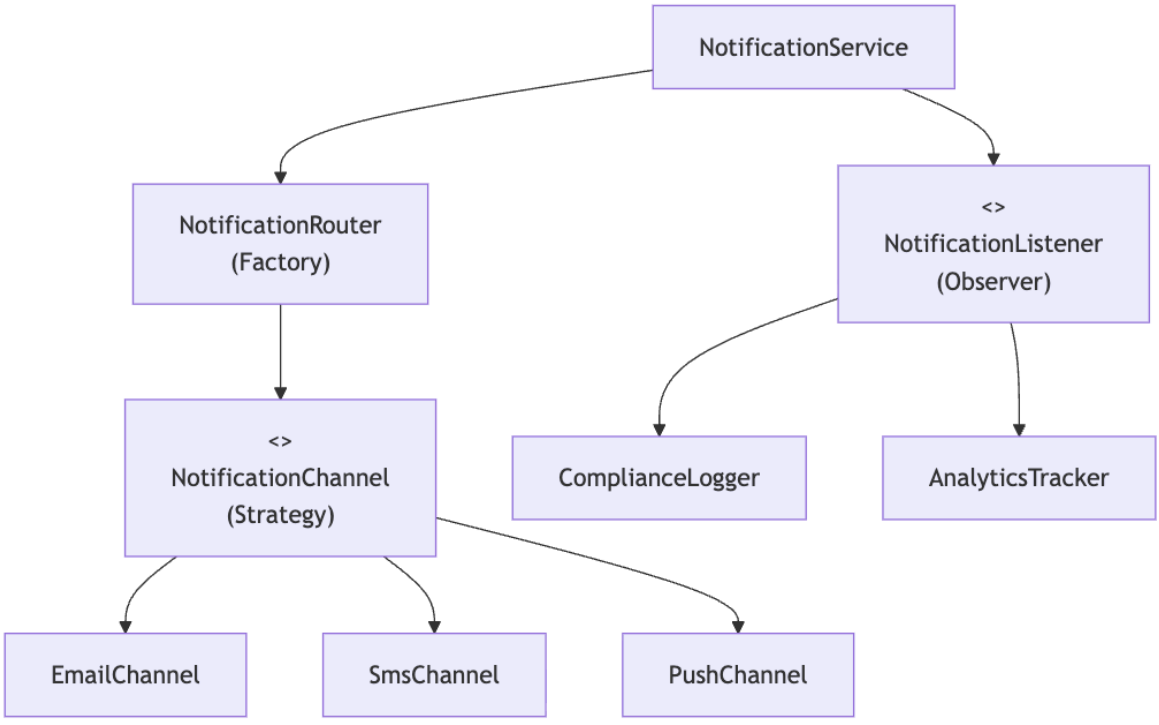
Step 4: Undo/Queuing/Pipeline

No undo needed. No pipeline of checks. No shared algorithm structure.

Step 5: Validate

Three patterns identified: Strategy (channels), Factory (routing), Observer (post-send reactions). Each solves a specific requirement. No pattern is added speculatively.

The Result



Three patterns, each justified by a specific requirement, with no over-engineering.

Interview Relevance: This walkthrough is exactly what interviewers want to see: a structured approach that maps requirements to patterns with explicit justification, not a memorized pattern list applied to every problem.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 83: Complete Signal-to-Pattern Mapping

This is the master reference table. Every pattern from Sections 4, 5, and 6 with its recognition signal and one-line justification.

Creational Patterns

Signal	Pattern	Justification
"Create different types based on configuration"	Factory Method	Centralize creation, decouple callers from concrete types
"Create a family of objects that must be consistent"	Abstract Factory	Guarantee all objects belong to the same family
"Object has 5+ parameters, many optional"	Builder	Self-documenting construction, validation in build()
"Exactly one instance required for correctness"	Singleton (prefer DI)	Connection pools, config registries

Structural Patterns

Signal	Pattern	Justification
"Integrate with an external API that has a different interface"	Adapter	Translate between incompatible interfaces
"Add logging/caching/metrics without modifying the class"	Decorator	Stackable behaviors, linear class growth
"Coordinate 3+ services behind a simple API"	Facade	Centralized sequencing and error recovery
"Cache results / lazy-load / check permissions"	Proxy	Control access transparently

Behavioral Patterns

Signal	Pattern	Justification
“Support multiple algorithms for X”	Strategy	Swap algorithms at runtime via interface
“When X happens, notify Y and Z”	Observer	Decouple event source from listeners
“Support undo/redo or queue operations”	Command	Actions as first-class objects
“Object behaves differently in each state”	State	Replace if/else on state with state objects
“Process through a pipeline of checks”	Chain of Responsibility	Sequential handlers with short-circuiting
“Same algorithm structure, different steps”	Template Method	Fixed skeleton, customizable sub-steps

Interview Tip: You do not need to memorize pattern names to succeed in an LLD interview. You need to recognize the signal in the requirements and know the structural solution. If you hear “support multiple types of X” and instinctively extract an interface with one implementation per type, you have applied Strategy, whether you name it or not.

Interview Relevance: This table is the equivalent of an algorithm pattern recognition table. Internalizing these signals is the single highest-leverage preparation for LLD interviews.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 84: Worked Example: E-Commerce Checkout System

Requirements

Design the checkout flow for a platform like Shopify. The system must:

- Support multiple payment methods (credit card, bank transfer, digital wallet)
- Apply discount codes with different discount types (percentage, flat amount, buy-one-get-one)

- Validate the order through a series of checks (inventory, payment method validity, shipping address, fraud detection)
- Process post-checkout actions (send confirmation email, update inventory, generate invoice)
- The invoice generator is expensive to initialize and not needed for every order (some orders are digital-only)

Pattern Selection Walkthrough

Requirement	Signal	Pattern	Justification
Multiple payment methods	"Different types of X"	Strategy (PaymentProcessor)	Each payment method is a separate algorithm
Select payment method based on order	"Create based on configuration"	Factory (PaymentProcessorFactory)	Factory decides which processor to use
Multiple discount types	"Different algorithms for X"	Strategy (DiscountStrategy)	Each discount type is a separate calculation
Series of validation checks	"Pipeline of checks"	Chain of Responsibility	Each check can short-circuit the flow
Post-checkout actions	"When X happens, do Y, Z, W"	Observer (CheckoutEventListener)	Decouple checkout from post-actions
Expensive invoice generator	"Expensive resource, not always needed"	Virtual Proxy	Lazy-load on first use

What Is NOT a Pattern

The checkout orchestration (sequence of steps) is a method in the `CheckoutService`. Not every piece of coordination needs a Facade. If only one caller performs the checkout, a well-structured method is sufficient. Patterns solve extensibility and decoupling problems, not sequencing problems.

Interview Relevance: This worked example demonstrates selective pattern application. Six requirements map to six patterns. The seventh consideration

(checkout orchestration) explicitly does not get a pattern. This selectivity is what earns a “strong hire” signal.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 84B: Worked Example: Chat Messaging System

Requirements

Design the messaging subsystem for a platform like Slack. The system must:

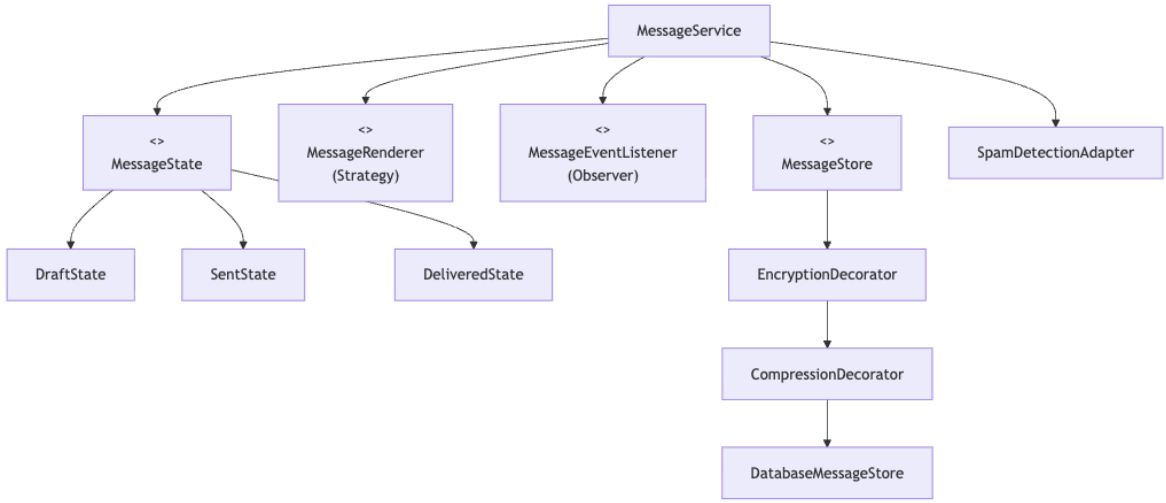
- Support multiple message types (text, image, file, code snippet) with different rendering
- Messages transition through states (draft, sent, delivered, read, deleted)
- When a message is sent, notify the recipient, update the unread count, and index for search
- Messages can be edited and deleted, with full edit history
- The system integrates with a third-party spam detection API that has a proprietary interface
- Message content should be optionally encrypted and compressed before storage

Pattern Selection Walkthrough

Requirement	Signal	Pattern
Multiple message types with different rendering	“Different types of X”	Strategy (MessageRenderer) + Factory
Message state transitions (draft, sent, delivered)	“Object behaves differently by state”	State
Post-send notifications (recipient, unread count, search index)	“When X happens, notify Y, Z, W”	Observer
Edit history with undo capability	“Store and reverse operations”	Command

Requirement	Signal	Pattern
Third-party spam API with proprietary interface	“Integrate with incompatible API”	Adapter
Optional encryption and compression	“Add behavior without modifying core”	Decorator

The Architecture



Six patterns, each solving a distinct requirement. The Decorator chain for storage (encryption wrapping compression wrapping database) is independent of the State machine for message lifecycle. Each pattern is isolated and testable.

Interview Relevance: A chat messaging system is one of the most common LLD interview questions at companies like Meta and Slack. Having a pre-practiced pattern mapping for this domain saves valuable interview time.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 85: Worked Example: File Storage and Processing Service

Requirements

Design a file storage service like Dropbox. The system must:

- Support uploading, downloading, and deleting files
- Files pass through a processing pipeline: virus scan, metadata extraction, thumbnail generation (for images), and indexing for search
- Processing steps vary by file type (images get thumbnails, documents get text extraction, videos get transcoding)
- Large files should be lazy-loaded (metadata first, content on demand)
- The system integrates with multiple cloud storage backends (AWS S3, Google Cloud Storage, Azure Blob Storage)
- All operations should be logged for audit compliance

Pattern Selection

Requirement	Pattern	Why Not an Alternative
Processing pipeline with sequential steps	Chain of Responsibility	Not Strategy, because multiple steps run in sequence, not one algorithm selected
Steps vary by file type	Template Method (or Strategy per step)	Template Method if the sequence is fixed, Strategy if steps are independently swappable
Lazy-loaded file content	Virtual Proxy	Not Decorator, because this is access control (when to load), not behavior addition
Multiple cloud storage backends	Adapter	Each backend has a different SDK; Adapter translates to your common interface
Audit logging on all operations	Decorator	Wraps the storage interface; independently removable
Upload/download/delete operations	No pattern needed	Simple interface methods

Key Insight: Not Every Requirement Needs a Pattern

The core CRUD operations (upload, download, delete) are methods on a `FileStorageService` interface. They do not need a pattern. The Adapter wraps the

cloud backends behind this interface. The Decorator adds logging on top. But the operations themselves are just methods. Wrapping simple method calls in Command objects or Strategy objects when there is only one algorithm is over-engineering.

Interview Tip: After identifying which patterns you will use, explicitly state which requirements do NOT get a pattern. “Upload and download are simple interface methods. I did not add a pattern here because there is no extensibility requirement and no variation to manage.” This demonstrates the restraint that separates senior designers from pattern-enthusiastic juniors.

Interview Relevance: File storage is a common LLD question at companies like Dropbox and Google. The key differentiator is the processing pipeline design (Chain of Responsibility vs Template Method) and the lazy-loading decision (Virtual Proxy).

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 85B: Interview Practice: Rapid Pattern Recognition

Instructions: For each requirement below, identify the primary pattern in under 10 seconds. Pause after each one and check your answer.

Requirement 1: “Users can export reports in PDF, CSV, or Excel format.”

Answer: Strategy (ExportStrategy). Three algorithms for the same operation, selected based on user choice.

Requirement 2: “When a payment fails, send an alert to the operations team, retry the payment in 5 minutes, and update the customer’s dashboard.”

Answer: Observer. Three independent reactions to a single event. The payment system should not know about ops alerts, retries, or dashboards.

Requirement 3: “The search query object has: search term (required), filters (optional, 0-N), sort order (optional), page size (optional, default 20), date range (optional), and include-archived flag (optional, default false).”

Answer: Builder. Six parameters, five optional, with defaults.

Requirement 4: "We need to integrate with a legacy inventory system that uses XML-RPC while our system uses REST-style JSON interfaces."

Answer: Adapter. Two incompatible interfaces that need to work together.

Requirement 5: "Add request timing, authentication logging, and rate limiting to all API endpoints without modifying the endpoint handlers."

Answer: Decorator (or Chain of Responsibility). Three cross-cutting concerns layered on top of existing handlers. Decorator if they wrap the handler; Chain of Responsibility if they form a pipeline.

Requirement 6: "A support ticket moves through states: Open, In Progress, Waiting on Customer, Resolved, Closed. Each state allows different actions."

Answer: State. Object behavior changes based on lifecycle state, with explicit valid transitions.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 86: Over-Engineering Anti-Pattern

Over-engineering is the most common anti-pattern in LLD interviews. It occurs when a candidate applies patterns, abstractions, or extensibility mechanisms that are not justified by the current requirements. Over-engineering signals to the interviewer that the candidate cannot distinguish between "what might change someday" and "what the requirements actually demand."

What Over-Engineering Looks Like

A candidate designing a parking lot system creates: a `VehicleFactory` (there are only 3 types, a simple enum works), a `ParkingStrategyFactory` that creates `ParkingStrategy` objects (there is only one strategy: find the nearest spot), a `SpotObserver` that fires events when spots are taken (no one subscribes to these events), and a `SpotStatePattern` for each spot (spots are either available or occupied, a boolean suffices).

Four patterns applied, none justified by requirements. The design is harder to understand, harder to implement, and harder to maintain than a straightforward solution with enums, simple classes, and boolean fields.

The Root Cause

Over-engineering usually comes from one of three mindsets:

1. “I want to show the interviewer that I know many patterns” (interviewers penalize this, not reward it)
2. “What if the requirements change in the future?” (design for today’s requirements, not hypothetical future ones)
3. “This is how I was taught to do it” (textbooks teach patterns in isolation; production code uses them selectively)

Interview Relevance: Interviewers specifically watch for over-engineering. A design with 2 well-justified patterns scores higher than a design with 6 unjustified patterns. Less is more when each choice is deliberate.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 86B: Minimum Viable Design Principle

The minimum viable design (MVD) principle states: apply the fewest patterns and abstractions that satisfy the stated requirements and make the design testable. Add more only when the interviewer explicitly asks for extensibility or when a requirement demands it.

The MVD Process

1. Start with plain classes, simple constructors, and direct method calls
2. Extract an interface only when there are (or the requirements state there will be) multiple implementations
3. Apply a pattern only when you can name the specific requirement it satisfies
4. If the interviewer says “what if we need to add X?”, then apply the pattern that enables X

MVD Applied: Parking Lot

Component	MVD Approach	Over-Engineered Approach
Vehicle types	`enum VehicleType { CAR, MOTORCYCLE, TRUCK }`	`VehicleFactory` + `Vehicle` interface + 3 implementations
Parking logic	`findNearestSpot()` method	`ParkingStrategy` interface + `NearestSpotStrategy` (single implementation)
Spot status	`boolean isOccupied`	`SpotState` interface + `OccupiedState` + `AvailableState`
Notifications	Not required, do not build	`SpotObserver` interface + event system

The MVD approach has ~4 classes. The over-engineered approach has ~15 classes solving problems that do not exist.

When MVD Does Not Apply

If the requirements explicitly state: "support multiple parking strategies" or "notify when a floor is full," then the corresponding patterns are justified. MVD means matching your design to the requirements, not minimizing patterns for their own sake.

Interview Tip: After presenting your design, tell the interviewer: "This design satisfies the stated requirements with [N] patterns. If you would like me to add extensibility for [specific scenario], I can show how [specific pattern] would enable that." This gives the interviewer the option to probe deeper while demonstrating that your initial design is intentionally minimal.

Interview Relevance: The MVD approach is what senior engineers practice in production. Interviewers reward it because it demonstrates the judgment to apply patterns selectively rather than reflexively.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 87: Patterns Interviewers Test Frequently vs Rarely

Not all patterns are equally likely to appear in LLD interviews. Focusing your preparation on high-frequency patterns gives you the best return on study time.

Pattern Frequency by Interview Likelihood

Tier	Patterns	Why
Tier 1: Almost always tested	Strategy, Observer, Factory Method, State	Core to every LLD problem. “Multiple types,” “event notification,” “object lifecycle” appear in nearly every question.
Tier 2: Frequently tested	Builder, Decorator, Command, Facade, Chain of Responsibility	Appear when the problem involves complex construction, cross-cutting concerns, undo/redo, or multi-service coordination.
Tier 3: Occasionally tested	Adapter, Proxy, Template Method, Abstract Factory	Appear in specific domains (legacy integration, caching, data pipelines, themed UIs).
Tier 4: Rarely tested directly	Singleton, Flyweight, Mediator, Memento, Visitor, Bridge	Either too simple (Singleton), too niche (Visitor), or covered by other patterns in practice.

Where to Focus

Tier 1 patterns appear in 80%+ of LLD interviews. If you deeply understand Strategy, Observer, Factory Method, and State, and you can recognize their signals instantly, you will handle the majority of LLD rounds.

Tier 2 patterns appear in 40-60% of interviews. They are the patterns that differentiate a “hire” from a “strong hire” because they solve secondary requirements (undo, logging, validation pipelines) that many candidates overlook.

Tiers 3 and 4 are worth knowing conceptually but not worth deep preparation unless you have already mastered Tiers 1 and 2.

Interview Relevance: Spending 80% of your prep time on Tier 1 and Tier 2 patterns gives you the highest coverage across companies and levels.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 87B: Pattern Testing Frequency by Company and Level

Different companies and levels emphasize different aspects of LLD. Knowing this helps you allocate preparation time.

By Company Type

Company Type	LLD Focus	Most Tested Patterns
E-commerce (Amazon, Shopify)	Order lifecycle, payment processing, inventory	State, Strategy, Observer, Facade, Factory
Social media (Meta, LinkedIn)	Feed systems, notifications, content moderation	Observer, Strategy, Chain of Responsibility, Decorator
Infrastructure (Google, Microsoft)	Data pipelines, storage systems, API design	Template Method, Adapter, Proxy, Chain of Responsibility
Ride-sharing (Uber, Lyft)	Pricing, matching, state machines	Strategy, State, Observer, Factory
Fintech (Stripe, Square)	Payment workflows, fraud detection, audit trails	State, Command, Chain of Responsibility, Observer

By Level

Level	Interviewer Expectations
Mid-level (L4/L5)	Correctly apply 2-3 patterns. Demonstrate OOP and SOLID understanding.
Senior (L5/L6)	Select patterns deliberately with trade-off justification. Spot anti-patterns. Combine patterns effectively.
Staff+ (L7+)	Minimal viable design. Explain why you chose NOT to use a pattern. Connect class-level design to system-level architecture.

Interview Tip: At mid-level, showing you know the patterns is sufficient. At senior level and above, showing restraint (not using a pattern when it is unnecessary) and connecting your class design to broader system concerns (scalability, team ownership, deployment boundaries) is what earns top ratings.

Interview Relevance: Calibrate your preparation to your target level. Mid-level candidates should focus on pattern mechanics. Senior candidates should focus on pattern selection and anti-pattern recognition.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 88: God Class Anti-Pattern

The God Class (also called a Blob class) is a single class that has grown to handle too many responsibilities. It typically has 20+ methods across unrelated domains, 10+ dependencies injected through its constructor, and is the class that every developer dreads modifying because changes to any feature risk breaking unrelated features.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

How God Classes Form

God Classes rarely start as God Classes. They start as a reasonable service class with 3-4 methods. As the product evolves, developers add methods to the existing class rather than creating new classes because it is faster and avoids the "where does this go?" decision. After 18 months of incremental additions, the class has 1500 lines, 12 dependencies, and 25 methods spanning authentication, billing, notifications, and reporting.

Detection Signals

1. Class has methods from 3+ unrelated domains
2. Constructor takes 8+ dependencies
3. Class is modified in every sprint by different developers for different features
4. Unit tests require mocking 10+ dependencies
5. Class name is vague: `UserService`, `AppManager`, `CoreHandler`

Interview Relevance: Interviewers present God Classes and ask "what is wrong with this design?" The answer is SRP violation. The refactoring is to identify change drivers and extract each into a focused class.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 88B: God Class: Concrete Example and Refactoring

Before: The God OrderService

```
public class OrderService {
    private final OrderRepository orderRepository;
    private final PaymentGateway paymentGateway;
    private final InventoryService inventoryService;
    private final ShippingService shippingService;
    private final EmailClient emailClient;
    private final SmsGateway smsGateway;
    private final AnalyticsTracker analyticsTracker;
    private final FraudDetector fraudDetector;
    private final TaxCalculator taxCalculator;
    private final DiscountEngine discountEngine;
    private final InvoiceGenerator invoiceGenerator;
    private final AuditLogger auditLogger;

    public OrderResult placeOrder(OrderRequest request) { ...
}
    public void cancelOrder(String orderId) { ... }
    public RefundResult processRefund(String orderId) { ... }
    public void updateShippingAddress(String orderId, Address
address) { ... }
    public InvoiceDocument generateInvoice(String orderId) {
... }
    public void sendOrderConfirmation(String orderId) { ... }
    public void sendShippingNotification(String orderId) {
... }
    public List<Order> getOrderHistory(String customerId) {
... }
    public OrderAnalytics getOrderAnalytics(DateRange range)
{ ... }
    public FraudCheckResult checkForFraud(OrderRequest reques
t) { ... }
    public Money calculateTax(OrderRequest request) { ... }
```

```
    public Money applyDiscounts(OrderRequest request) { ... }
}
```

Twelve dependencies, twelve methods, six unrelated domains (ordering, payment, shipping, notifications, analytics, fraud).

After: Focused Services

```
public class OrderPlacementService {
    private final OrderRepository orderRepository;
    private final PaymentService paymentService;
    private final InventoryService inventoryService;
    private final OrderEventPublisher eventPublisher;

    public OrderResult placeOrder(OrderRequest request) {
        inventoryService.reserve(request.getItems());
        PaymentResult payment = paymentService.charge(reques
t);
        Order order = Order.create(request, payment.getTransa
ctionId());
        orderRepository.save(order);
        eventPublisher.publish(new OrderPlacedEvent(order));
        return OrderResult.success(order.getId());
    }
}
```

```
public class OrderPricingService {
    private final TaxCalculator taxCalculator;
    private final DiscountEngine discountEngine;

    public PricingBreakdown calculatePricing(OrderRequest req
uest) {
        Money subtotal = request.calculateSubtotal();
        Money discount = discountEngine.bestDiscount(reques
t);
        Money tax = taxCalculator.calculate(subtotal.subtract
```

```
(discount));
        return new PricingBreakdown(subtotal, discount, tax);
    }
}
```

Each service has 2-4 dependencies and 1-3 methods in a single domain. The `OrderPlacementService` orchestrates the core flow. Notifications are handled by Observer listeners on the `OrderPlacedEvent`. Analytics and fraud detection are separate services.

Interview Relevance: Refactoring a God Class into focused services is one of the most common LLD interview exercises. Showing the before and after with explicit dependency counts demonstrates practical refactoring skill.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 89: Feature Envy and Inappropriate Intimacy

Feature Envy

Feature Envy occurs when a method in one class uses more data from another class than from its own. The method is “envious” of the other class’s data and should probably live there instead.

```
public class OrderReportGenerator {
    public String generateSummary(Order order) {
        double total = order.getItems().stream()
            .mapToDouble(item -> item.getPrice() * item.getQu
antity())
            .sum();
        double tax = total * order.getTaxRate();
        double shipping = order.getWeight() > 10 ? 15.00 : 5.
00;
        return String.format("Total: $%.2f, Tax: $%.2f, Shipp
ing: $%.2f",
            total, tax, shipping);
```

```
        }
    }
```

This method accesses `items`, `price`, `quantity`, `taxRate`, and `weight` from `Order` but uses nothing from its own class. The calculation logic belongs in `Order` or a dedicated `OrderPricingService`.

The Fix: Move the method to the class whose data it uses, or create a service class that encapsulates the calculation.

Inappropriate Intimacy

Inappropriate Intimacy occurs when two classes are too tightly coupled, accessing each other's internal details. This often manifests as classes reaching into each other's private data through getters, or two classes that cannot be modified independently.

Detection Signals for Both

Smell	Signal
Feature Envy	A method calls 3+ getters on another object and uses 0 from its own
Inappropriate Intimacy	Two classes each have references to each other
Inappropriate Intimacy	A class accesses internal collections from another class and modifies them
Feature Envy	Moving the method to the other class would eliminate all the getter calls

Interview Relevance: Interviewers test Feature Envy by showing you a method and asking "is this in the right class?" If the method uses more data from another class, the answer is "this method should be moved to the class it envies."

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 90: Shotgun Surgery Anti-Pattern

Shotgun Surgery is the opposite of the God Class. While a God Class has too many responsibilities in one place, Shotgun Surgery scatters one responsibility across too many places. A single logical change requires editing 10+ files.

Example

Adding a new customer notification type requires editing:

- `CustomerService.java` (add notification logic)
- `NotificationTemplates.java` (add new template)
- `EmailFormatter.java` (add formatting for new type)
- `SmsFormatter.java` (add formatting for new type)
- `NotificationConfig.java` (add configuration entry)
- `NotificationValidator.java` (add validation for new type)
- `NotificationMetrics.java` (add metrics tracking)
- `NotificationTest.java` (add test cases)

Eight files modified for one logical change. If a developer forgets to update `SmsFormatter`, SMS notifications for the new type silently fail in production.

The Fix

Consolidate the scattered responsibility. Each notification type should be a self-contained class (a Strategy implementation) that encapsulates its template, formatting, validation, and configuration. Adding a new type means creating one new file, not editing eight existing ones.

```
public class WelcomeNotification implements NotificationType
{
    public String getTemplateName() { return "welcome"; }
    public String formatForEmail(Customer customer) { ... }
    public String formatForSms(Customer customer) { ... }
    public boolean isValid(NotificationRequest request) { ...
}
    public Map<String, String> getDefaultConfig() { ... }
}
```

One file per notification type. Adding a new type is one new class. This is the OCP principle applied to resolve Shotgun Surgery.

Interview Tip: Shotgun Surgery and God Class are two sides of the same coin. God Class means too much in one place. Shotgun Surgery means too little in each place. The fix for both is finding the right boundary: each class owns one complete responsibility, no more and no less.

Interview Relevance: Interviewers test Shotgun Surgery by describing a change that touches many files and asking "how would you restructure this?" The answer is always: consolidate the scattered responsibility into a cohesive class or module.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 91: Circular Dependencies

A circular dependency occurs when class A depends on class B, and class B depends on class A (directly or through a chain). Circular dependencies create systems where you cannot understand, test, or deploy one class without the other. They are a strong signal of a design flaw.

The Problem

```
public class OrderService {
    private final BillingService billingService;

    public void placeOrder(Order order) {
        billingService.createInvoice(order);
    }
}

public class BillingService {
    private final OrderService orderService;

    public void processPayment(String invoiceId) {
        Order order = orderService.getOrder(invoiceId);
```

```
            chargeCustomer(order);
        }
    }
```

`OrderService` depends on `BillingService` and `BillingService` depends on `OrderService`. You cannot instantiate either one without the other. You cannot test either one without a real (or mocked) instance of the other. If both classes are in different modules or services, you have a deployment cycle: neither can be deployed independently.

Three Fixes

Fix 1: Extract Shared Interface

Both classes depend on an interface that neither owns. `BillingService` depends on `OrderRepository` (an interface) instead of `OrderService` directly. The cycle is broken because the dependency arrow points to the interface, not to the concrete class.

Fix 2: Introduce an Event

`OrderService` publishes an `OrderPlacedEvent`. `BillingService` listens for it. Neither class references the other. The Observer pattern breaks the cycle by introducing an indirection through events.

Fix 3: Extract a Third Class

The shared logic lives in a new class (`OrderBillingCoordinator`) that depends on both `OrderService` and `BillingService`. Neither `OrderService` nor `BillingService` depends on the other. The coordinator manages the interaction.

Interview Relevance: Circular dependencies are a design smell that interviewers ask you to detect and resolve. The three fixes (shared interface, events, coordinator extraction) cover the most common interview scenarios.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 91B: Premature Abstraction

Premature abstraction occurs when you create interfaces, abstract classes, or pattern-based indirection before you have evidence that the abstraction is

needed. It is the abstraction equivalent of premature optimization: it adds complexity now for a benefit that may never materialize.

The Problem

```
public interface UserRepository {
    User findById(String id);
    void save(User user);
}

public class PostgresUserRepository implements UserRepository
{
    private final DataSource dataSource;

    public PostgresUserRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override
    public User findById(String id) {
        return JdbcTemplate.queryForObject(dataSource, "SELEC
T * FROM users WHERE id = ?", id);
    }

    @Override
    public void save(User user) {
        JdbcTemplate.update(dataSource, "INSERT INTO users VA
LUES (?, ?)", user.getId(), user.getName());
    }
}
```

If there is only one implementation and no plan to add another, the interface adds a layer of indirection with zero benefit. Every developer who navigates from the interface to the implementation wastes time. Every refactoring tool that renames a method must update the interface and the implementation. The interface exists

“just in case” the database changes, but the database has not changed in five years.

When Abstraction IS Justified

1. There are currently 2+ implementations (Strategy, Factory)
2. The requirement explicitly states that the component will be swapped (DIP for third-party integrations)
3. The abstraction enables testability that would otherwise be impossible (mocking external services)
4. Multiple consumers depend on the contract (the interface stabilizes the API)

When Abstraction Is Premature

1. Single implementation with no concrete plan for a second
2. The interface mirrors the implementation exactly with no behavioral contract
3. The abstraction is added “for flexibility” without a specific flexibility requirement
4. Adding the abstraction makes the codebase harder to navigate for new developers

The Guideline

Wait until you need the abstraction, then add it. Extracting an interface from an existing class takes 30 seconds with modern IDE tooling. There is no penalty for adding it later and a real cost to adding it too early.

Interview Relevance: In an LLD interview, do not add interfaces for every class. Add them where the requirements demand multiple implementations or where DIP is needed for testability. The interviewer will respect your restraint.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 92: Interview Practice: Spotting Anti-Patterns in Code

Scenario

An interviewer shows you this class and asks: “What design problems do you see?”

```
public class ReportManager {
    private final DatabaseConnection dbConnection;
    private final SmtpClient smtpClient;
    private final PdfRenderer pdfRenderer;
    private final CsvWriter csvWriter;
    private final SlackWebhook slackWebhook;
    private final S3Client s3Client;
    private final MetricsCollector metricsCollector;
    private final UserRepository userRepository;

    public Report generateReport(ReportRequest request) {
        List<DataRow> data = dbConnection.query(request.getSq
lQuery());
        Report report = new Report(data);
        metricsCollector.increment("reports_generated");
        return report;
    }

    public void emailReport(Report report, String recipientEm
ail) {
        byte[] pdf = pdfRenderer.render(report.getData());
        smtpClient.sendWithAttachment(recipientEmail, "Your R
eport", pdf);
    }

    public void uploadToS3(Report report, String bucketPath)
{
        byte[] csv = csvWriter.write(report.getData());
        s3Client.putObject(bucketPath, csv);
    }

    public void notifySlack(String channel, String message) {
        slackWebhook.post(channel, message);
```

```
    }

    public User getReportOwner(String reportId) {
        return userRepository.findById(reportId);
    }
}
```

Pause the video and identify all anti-patterns for 60 seconds before continuing.

Anti-Patterns Identified

Anti-Pattern	Evidence	Fix
God Class	8 dependencies, 5 methods across reporting, email, storage, messaging, and user lookup	Split into `ReportGenerator`, `ReportExporter`, `NotificationService`
Feature Envy	`getReportOwner` has nothing to do with reports, it just delegates to `userRepository`	Move to the caller or remove entirely
SRP Violation	Four change drivers: report logic, export formats, delivery channels, user lookup	Each change driver becomes its own class
DIP Violation	Direct dependency on `SmtpClient`, `S3Client`, `SlackWebhook` (concrete infrastructure)	Depend on `EmailSender`, `FileStorage`, `MessagingService` interfaces
Premature coupling	`notifySlack` is hardcoded to Slack	Should be a `NotificationChannel` interface

A strong answer identifies at least three of these anti-patterns, names the SOLID principle each violates, and sketches the refactored structure with focused classes and interfaces.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 93: How to Present Your Design in an Interview

Knowing the patterns is not enough. How you communicate your design in real time determines your interview score. This slide covers the presentation structure that earns the strongest evaluations.

The Five-Phase LLD Interview Structure

Phase 1: Clarify Requirements (3-5 minutes)

Ask clarifying questions to narrow scope. “What types of X do we need to support? Will there be more types in the future? Do we need undo support? How many concurrent users?” Do not design for every possible requirement. Design for the stated ones and ask about ambiguous ones.

Phase 2: Identify Core Objects (3-5 minutes)

List the 4-6 core classes and their responsibilities. State one sentence per class: “OrderService orchestrates the checkout flow. PaymentProcessor handles payment execution. NotificationService sends post-order notifications.” This establishes the vocabulary for the rest of the discussion.

Phase 3: Define Relationships and Interfaces (5-10 minutes)

Draw or describe the relationships between classes. Define key interfaces. This is where patterns emerge. “Since we need multiple payment types, PaymentProcessor is an interface with one implementation per type. Since notifications should not block checkout, NotificationService subscribes to order events through an Observer.”

Phase 4: Detailed Class Design (10-15 minutes)

Write key method signatures and constructors. Show one or two critical methods in detail. This is where encapsulation, SOLID, and clean code are evaluated.

Phase 5: Extensibility and Trade-offs (5 minutes)

Discuss what you would add if requirements changed. “If we needed undo support, I would introduce Command objects for each order action.” Proactively mention one trade-off you made: “I chose composition over inheritance for the pricing strategies because we have two independent dimensions of variation.”

Interview Relevance: This five-phase structure maps directly to how interviewers evaluate LLD candidates. Each phase corresponds to a rubric dimension: requirements clarification, object identification, OOP/SOLID compliance, design depth, and extensibility thinking.

SLIDE 93B: Structuring Your LLD Answer: Common Timing Mistakes

Mistake 1: Spending 15 Minutes on Requirements

Over-clarifying delays the design phase. Ask 3-5 targeted questions, state your assumptions for anything ambiguous, and start designing. The interviewer can redirect if your assumptions are wrong.

Mistake 2: Jumping Straight to Code

Writing code before establishing the class structure means you are designing while coding, which produces messy, unplanned results. Spend 5 minutes on the class diagram before writing any method.

Mistake 3: Drawing a Full UML Diagram with No Explanation

A diagram without verbal explanation leaves the interviewer guessing why you made each decision. Narrate as you draw: “This is an interface because we need multiple implementations. This class depends on the interface, not the concrete class, for testability.”

Mistake 4: Designing for Every Possible Future Requirement

“What if we need to support 50 notification channels and 12 payment providers and multilingual templates and...” Stop. Design for the stated requirements. Mention extensibility points at the end.

Recommended Time Allocation for a 45-Minute LLD Round

Phase	Time	What Interviewers Evaluate
Clarify requirements	3-5 min	“Do they ask smart questions?”
Core objects and responsibilities	3-5 min	“Do they identify the right abstractions?”
Interfaces and relationships	5-10 min	“Do they use OOP and SOLID correctly?”

Phase	Time	What Interviewers Evaluate
Detailed design and key methods	10-15 min	“Can they write clean, encapsulated code?”
Extensibility and trade-offs	5 min	“Do they think about future changes?”
Interviewer questions	5-10 min	“Can they defend their decisions?”

Interview Tip: Practice the timing with a stopwatch. Most candidates spend too long on clarification and run out of time before reaching the detailed design phase, which is where the strongest evaluation signals come from.

Interview Relevance: Time management in an LLD interview is a meta-skill that directly affects your score. A complete, well-paced answer with 2 patterns scores higher than a perfect but incomplete answer with 5 patterns.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 94: Note It Down: Complete Pattern Selection Reference

This is the master reference for pattern selection. Screenshot this slide or write it down.

The Three Questions

1. **What varies?** Answer maps to: Strategy, State, Factory, Template Method
2. **What reacts?** Answer maps to: Observer, Command, Chain of Responsibility
3. **What wraps?** Answer maps to: Decorator, Proxy, Adapter, Facade

The Danger Signals (Stop and Reconsider)

You Are About To...	Ask Yourself...
Create an interface with 1 implementation	“Is there actually a second implementation?”
Apply a pattern	“Which specific requirement does this solve?”
Add a Factory for 1 object type	“Is a constructor sufficient?”
Use Singleton	“Can I use dependency injection instead?”

You Are About To...	Ask Yourself...
Add a Decorator	“Is this a behavior addition (Decorator) or access control (Proxy)?”
Create 4+ patterns in one design	“Am I over-engineering?”
Use inheritance	“Is this truly is-a? Would composition be safer?”

The Anti-Pattern Quick Check

Smell	Anti-Pattern	Fix
Class with 8+ dependencies	God Class	Split by change driver
Method uses another class’s data more than its own	Feature Envy	Move method to the other class
One change touches 8+ files	Shotgun Surgery	Consolidate scattered responsibility
Class A depends on B, B depends on A	Circular Dependency	Extract interface, event, or coordinator
Interface with 1 implementation	Premature Abstraction	Remove interface until second implementation exists

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 95: Section 7 Interview Practice and Common Mistakes

Practice Scenario

You are in an LLD interview designing a task management system like Jira. The requirements are: tasks can be created, assigned, and commented on. Tasks move through states (To Do, In Progress, In Review, Done). When a task changes state, the assignee and watchers should be notified. Tasks can have different priority levels that affect sorting in the board view.

You have 45 minutes. Walk through your design.

Pause the video and plan your five-phase approach for 2 minutes before continuing.

Strong Approach

Phase 1 (3 min): Clarify. “How many task states? Are transitions restricted? What notification channels? Is priority fixed or configurable?”

Phase 2 (3 min): Core objects. `Task`, `TaskState`, `TaskBoard`, `NotificationService`, `PriorityComparator`.

Phase 3 (7 min): Relationships. Task lifecycle = State pattern. Notifications on state change = Observer. Priority sorting = Strategy (if configurable) or simple Comparator (if fixed). No Factory needed since there is one task type. No Builder needed since tasks have few required fields.

Phase 4 (15 min): Detail the `TaskState` interface and two state implementations. Show `TaskEventListener` interface with `EmailNotifier` and `SlackNotifier`. Write `Task.transition()` method.

Phase 5 (5 min): “If we need undo for state transitions, I would wrap transitions in Command objects. If we need custom workflows per project, I would make the state machine configurable.”

Common Pattern Selection Mistakes

Mistake	Why It Is Wrong	Stronger Approach
Using every pattern in one design	Over-engineering makes the design harder to understand	Use only patterns justified by requirements
Cannot explain why a pattern was chosen	Suggests pattern memorization, not understanding	Name the requirement, then the pattern it demands
Applying Strategy when State is correct	Confusing “algorithm selection” with “lifecycle behavior”	Use the transition test: does the object change behavior over time?
Ignoring anti-patterns in their own design	Missing a God Class or Feature Envy in the solution	Review your own design for the six anti-patterns
Spending 20 minutes on requirements	Leaves insufficient time for design and code	3-5 focused questions, then design
Naming patterns without applying them	“I would use Strategy and Observer” with no code	Show the interfaces, implementations, and wiring

Interview Relevance: This scenario tests pattern selection restraint. The correct answer uses 2-3 patterns (State, Observer, and possibly Strategy). Using more

than that is a red flag for over-engineering.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 96: Section Summary and Key Takeaways

Concepts Covered

Concept	One-Line Summary
Pattern Selection Process	Five steps: identify objects, dimensions of variation, cross-cutting concerns, undo/queue/pipeline needs, then validate
Signal-to-Pattern Mapping	Requirement signals map to specific patterns across creational, structural, and behavioral categories
Over-Engineering	Applying patterns without requirements justification; penalized, not rewarded
Minimum Viable Design	Fewest patterns that satisfy stated requirements and enable testability
God Class	Too many responsibilities in one class; fix by splitting along change drivers
Feature Envy	Method uses another class's data; fix by moving method
Shotgun Surgery	One change touches many files; fix by consolidating scattered responsibility
Circular Dependencies	A depends on B depends on A; fix with interface extraction, events, or coordinator
Premature Abstraction	Interface with one implementation; defer until second implementation exists
Interview Presentation	Five-phase structure with explicit time allocation

Common Interview Mistakes

Mistake	Stronger Approach
"I used 6 patterns to make the design extensible"	"I used 2 patterns, each justified by a specific requirement"

Mistake	Stronger Approach
“I added interfaces for every class”	“I added interfaces where the requirements demand multiple implementations”
“I know all the patterns” (lists them)	“I hear this requirement and it maps to this pattern because...”
“I jumped straight to coding”	“Let me start with the core objects and their responsibilities”
“I spent 15 minutes clarifying”	“I asked 3 targeted questions and stated my assumptions”

How This Connects to Other Sections

- **Sections 2-3 (OOP + SOLID):** Pattern selection is justified by OOP principles and SOLID violations. “I chose Strategy because the if/else chain violates OCP” is the connection.
- **Sections 4-6 (All Patterns):** This section synthesized the individual patterns into a unified selection framework with signal mapping and worked examples.
- **Section 8 (Cheat Sheet):** The master reference table from this section is compiled with all other section references into the final course cheat sheet.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn